DAY 18 | HANDS-ON ACTIVITIES

Git, CI/CD and Docker: Problem Statements

Seven problems. Git works in Colab and VS Code. Docker commands need Docker Desktop (VS Code) or an Azure DevOps agent.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: First Repository

Level: Warm-up | **Concepts:** init, add, commit, log

Create a repo for your Task API, make three commits with clear messages, and show the history with git log --oneline.

Sample run / expected result

```
a1b2c3d Add delete endpoint
9f8e7d6 Add task model
1a2b3c4 Initial FastAPI app
```

Hints

- git config user.name and user.email first.

Problem 2: Feature Branch

Level: Easy | **Concepts:** switch, merge

Create feature/health-check, add a /health endpoint, commit, switch to main, merge, and delete the branch.

Sample run / expected result

```
Fast-forward
 app/main.py | 4 ++++
Deleted branch feature/health-check
```

Hints

- git switch -c feature/health-check
- git branch -d feature/health-check

Problem 3: Resolve a Conflict

Level: Medium | **Concepts:** merge conflicts

Change the same line on two branches, merge them, resolve the conflict by keeping a combined version, and commit.

Sample run / expected result

```
CONFLICT (content): Merge conflict in greeting.py
... after fix ...
[main 7c1d2e3] Resolve greeting conflict
```

Hints

- Search the file for <<<<<<< and >>>>>>> markers.

Problem 4: Push and Pull Request

Level: Medium | **Concepts:** Azure Repos, PR

Create a repo in Azure Repos, add it as origin, push main and a feature branch, then open a pull request with a clear description and one reviewer.

Sample run / expected result

```
git remote add origin https://dev.azure.com/<org>/<project>/_git/tasks-api
git push -u origin main
```

Hints

- Use a personal access token or Git Credential Manager when prompted.

Problem 5: Multi-stage Pipeline

Level: Challenging | **Concepts:** stages, dependsOn, condition

Write azure-pipelines.yml with a Test stage (lint and pytest with published results) and a Docker stage that depends on Test and only runs on main.

Sample run / expected result

```
Stages: Test -> Docker
Docker skipped on feature branches
```

Hints

- Validate structure with yaml.safe_load before pushing.

Problem 6: Write the Dockerfile

Level: Medium | **Concepts:** Dockerfile, .dockerignore

Write a Dockerfile and .dockerignore for the Task API. Build it, run it on port 8000, and call /health.

Sample run / expected result

```
docker run -d -p 8000:8000 tasks-api:1.0
curl localhost:8000/health -> {"status":"ok"}
```

Hints

- Copy requirements before code for faster rebuilds.

Problem 7: Smaller and Safer Image

Level: Stretch | **Concepts:** slim images, non-root user, caching

Improve the image: use python:3.11-slim, run as a non-root user, and add a HEALTHCHECK. Compare docker images sizes before and after, and time a rebuild after a code-only change.

Sample run / expected result

```
tasks-api:1.0  1.1GB
tasks-api:1.1  180MB
Rebuild after code change: 3 s
```

Hints

- docker images
- RUN useradd -m appuser USER appuser

Ganbatte (頑張って): do your best